# Task Assignment and Roles: Real-Time IoT Data Pipeline

## 1. Team Structure

| # | Member | Role | Main Responsibility |
|---|---|---|---|
| 1 | Rana Shaaban Abdelqader Mohamed | Team Leader and Dashboard Developer | Coordination, planning, dashboard, final integration |
| 2 | Abdelrahman Sayed Taha Morsi | Kafka Consumer Developer | Kafka consumer and batched files for the batch path |
| 3 | Mostafa Ahmed Mahmoud Shaban | Simulation and Streaming Developer | Data simulator, streaming consumer, and alerts |
| 4 | Shahd Safwat Mohamed | ETL Developer | ETL pipeline into Snowflake and data validation |
| 5 | Waleed Hamouda Abd Elfattah Mahmoud | API and Producer Developer | Flask API and Kafka producer |

## 2. Detailed Responsibilities

### Rana Shaaban Abdelqader Mohamed (Team Leader, Dashboard)

- Lead the team, run weekly meetings, and track progress against the Project Plan.
- Make sure the data schema and interfaces between members are agreed.
- Build the real-time dashboard (sensor trends, anomaly counts, alert history).
- Connect the dashboard to the processed data and the alert stream.
- Lead end-to-end integration testing and compile the final report and presentation.
- Make sure all documents and code are uploaded to GitHub.

### Waleed Hamouda Abd Elfattah Mahmoud (Flask API and Kafka Producer)

- Build the Flask API that receives sensor readings.
- Build the Kafka producer that publishes readings to the Kafka topic.

- Set up the Kafka topic(s) with the team.
- Handle errors and invalid messages at the ingestion point.
- Document the API and producer.

### Mostafa Ahmed Mahmoud Shaban (Data Simulator, Streaming and Alerts)

- Define the data schema (sensor ID, timestamp, temperature, humidity, location) with the team.
- Build the Python simulator that generates readings every 5 seconds and test that the output is realistic and consistent.
- Build a separate Kafka consumer for the streaming path.
- Define threshold rules and implement the alert logic.
- Test alerts using simulated anomalies and document the streaming architecture.

### Abdelrahman Sayed Taha Morsi (Kafka Consumer and Batched Files)

- Build the Kafka consumer for the batch path.
- Write incoming messages to batched files (for example CSV) for the ETL to pick up.
- Make the consumer reliable under continuous data flow.
- Monitor pipeline performance during streaming tests.
- Document the consumer and the batch file format.

### Shahd Safwat Mohamed (ETL to Snowflake)

- Set up the Snowflake database, schema, and tables.
- Build the ETL: extract batched files, transform (clean, flag anomalies, compute averages), and load into Snowflake.
- Validate the loaded data for completeness and accuracy and write the validation report.
- Optimize the ETL script for performance.
- Document the ETL pipeline design.

# 3. Task Ownership by Week

| Week | Rana | Abdelrahman | Mostafa | Shahd | Waleed |
|---|---|---|---|---|---|
| 1 | Planning, environment coordination | Start consumer design | Schema and simulator | Snowflake setup | Flask API and Kafka producer |
| 2 | Dashboard design | Consumer and batched files | Simulator support and testing | ETL to Snowflake and validation | Producer fixes and support |
| 3 | Dashboard development | Performance monitoring | Streaming consumer and alerts | Validation fixes | Support alert testing |
| 4 | Dashboard, integration, report, presentation | Testing and own documentation | Testing and own documentation | Testing and own documentation | Testing and own documentation |

# 4. Responsibility Matrix

R = Responsible (does the work), A = Accountable (final owner), C = Consulted, I = Informed.

| Component | Rana | Abdelrahman | Mostafa | Shahd | Waleed |
|---|---|---|---|---|---|
| Data schema | A | C | R | C | C |
| Data simulator | I | I | R / A | I | C |
| Flask API | I | I | C | I | R / A |
| Kafka producer | I | C | C | I | R / A |
| Kafka consumer (batch) | I | R / A | C | C | C |
| ETL to Snowflake | I | C | I | R / A | I |
| Streaming consumer and alerts | I | C | R / A | I | C |
| Dashboard | R / A | C | C | C | I |
| Integration testing | A | R | R | R | R |
| Final report and presentation | A / R | R | R | R | R |

## 5. Hand-offs Between Members

1. **Mostafa to Waleed:** simulator output format (schema) so the API accepts it.
2. **Waleed to Abdelrahman and Mostafa:** Kafka topic name and message format.
3. **Abdelrahman to Shahd:** batched files location and file format.
4. **Shahd to Rana:** Snowflake tables for the dashboard.
5. **Mostafa to Rana:** alert output (topic or table) for the dashboard.

## 6. Team Rules

- Each member documents their own component in a short README inside their folder on GitHub.
- Changes that affect another member's work (schema, topic names, file formats) are agreed with that member first.
- Blockers are reported to the team leader as soon as they appear.
- Code is pushed to GitHub regularly, using branches and short commit messages.